Universidad de los Andes

Departamento de Ingeniería Industrial · Modelado de Sistemas bajo Incertidumbre

Fundamentos de Simulación Estocástica: De la Congruencia Lineal a la Transformada Inversa

Alejandra Tabares Pozos

Índice

Resumen

Esta lectura técnica introduce los mecanismos fundamentales para la generación de variables aleatorias en computadoras digitales. Se explora la paradoja de generar números aleatorios mediante algoritmos deterministas (Generadores Congruenciales Lineales) y se detalla cómo transformar estos números uniformes en distribuciones de probabilidad específicas mediante el Método de la Transformada Inversa. El documento integra teoría matemática rigurosa con ejemplos numéricos secuenciales y su implementación computacional en Python.

1. Introducción

En la ingeniería moderna, ya sea modelando una línea de producción industrial o la volatilidad de un activo financiero, nos enfrentamos a la incertidumbre. Para replicar esta incertidumbre en una computadora, necesitamos una fuente de números que se comporten como variables aleatorias.

Sin embargo, las computadoras son máquinas deterministas; dada una misma entrada, siempre producen la misma salida. ¿Cómo generamos azar? La respuesta radica en los **Números Pseudoaleatorios**. Estos son sucesiones de números generados algorítmicamente que, aunque deterministas, satisfacen pruebas estadísticas de aleatoriedad (independencia y uniformidad).

El proceso de simulación estocástica sigue un flujo lógico de dos etapas:

1. **Generación de la Fuente (Materia Prima):** Producir números uniformemente distribuidos en el intervalo $(0, 1)$, denotados como $R \sim U(0, 1)$.
2. **Generación de la Variada (Producto Final):** Transformar R en una variable X que siga una distribución específica (Exponencial, Normal, Weibull, etc.).

2. Generadores Congruenciales Lineales (LCG)

El algoritmo más clásico y pedagógico para la primera etapa es el Generador Congruencial Lineal, propuesto por D.H. Lehmer. Su elegancia radica en su simplicidad y bajo costo computacional.

2.1. El Algoritmo

Un LCG genera una secuencia de números enteros $Z_1, Z_2, \ldots$ mediante la relación recursiva:

$$Z_i = (aZ_{i-1} + c) \pmod{m} \tag{1}$$

Donde los parámetros enteros son:

- $m > 0$: El **módulo**. Determina el valor máximo posible y la longitud máxima del ciclo (periodo).
- a $(0 < a < m)$: El **multiplicador**.
- c $(0 \leq c < m)$: El **incremento**.

- Z_0 $(0 \leq Z_0 < m)$: La **semilla** o valor inicial.

Para obtener el número pseudoaleatorio uniforme $R_i \in (0, 1)$, normalizamos el entero generado:

$$R_i = \frac{Z_i}{m} \tag{2}$$

2.2. Ejemplo Numérico 1: El Mecanismo del LCG

Propósito: Ilustrar la mecánica recursiva y el concepto de periodo.

Supongamos un generador didáctico "de juguete" con los siguientes parámetros:

$$m = 16, \quad a = 5, \quad c = 3, \quad Z_0 = 7$$

Generemos los primeros valores de la secuencia:

1. **Iteración** $i = 1$:

$$Z_1 = (5 \times 7 + 3) \quad (\text{mód } 16) = 38 \quad (\text{mód } 16)$$

$$38 = 2 \times 16 + \mathbf{6} \quad \Rightarrow \quad Z_1 = 6$$

$$R_1 = \frac{6}{16} = \mathbf{0.3750}$$

2. **Iteración** $i = 2$:

$$Z_2 = (5 \times 6 + 3) \quad (\text{mód } 16) = 33 \quad (\text{mód } 16)$$

$$33 = 2 \times 16 + \mathbf{1} \quad \Rightarrow \quad Z_2 = 1$$

$$R_2 = \frac{1}{16} = \mathbf{0.0625}$$

3. **Iteración** $i = 3$:

$$Z_3 = (5 \times 1 + 3) \quad (\text{mód } 16) = 8 \quad (\text{mód } 16) = \mathbf{8}$$

$$R_3 = \frac{8}{16} = \mathbf{0.5000}$$

Observación: Si continuamos iterando, eventualmente el valor Z_i volverá a ser 7, reiniciando el ciclo. En simulación profesional, usamos m muy grandes (ej. $2^{31} - 1$) para evitar que el patrón se repita durante la simulación.

3. Método de la Transformada Inversa

Una vez que tenemos nuestra fuente de aleatoriedad $R \sim U(0, 1)$, necesitamos modelar fenómenos reales. Por ejemplo, los tiempos entre llegadas de clientes a un banco no son uniformes; suelen seguir una distribución Exponencial. ¿Cómo convertimos R en X?

3.1. Fundamento Teórico

El método se basa en la propiedad de la Función de Distribución Acumulada (CDF), $F(x)$, la cual mapea valores del dominio de la variable aleatoria al intervalo $[0, 1]$. Si X es una variable aleatoria continua con CDF $F(x)$, entonces la variable aleatoria definida como $U = F(X)$ sigue una distribución uniforme $U(0, 1)$.

Por consiguiente, si invertimos la función:

$$X = F^{-1}(R) \quad \text{donde } R \sim U(0, 1) \tag{3}$$

La variable resultante X tendrá la distribución F.

3.2. Ejemplo Numérico 2: Simulación de Tiempos de Servicio

Propósito: Conectar el LCG con la Transformada Inversa en un problema aplicado.

Contexto: Un ingeniero industrial está simulando un cajero automático. Se ha determinado que el tiempo de servicio X (en minutos) sigue una distribución **Exponencial** con una tasa media de servicio de $\mu = 0.5$ minutos/cliente (o parámetro de tasa $\lambda = 1/\mu = 2$).

Paso 1: Derivar la fórmula generadora. La PDF es $f(x) = \lambda e^{-\lambda x}$ para $x \geq 0$. La CDF es:

$$F(x) = \int_0^x \lambda e^{-\lambda t} dt = 1 - e^{-\lambda x}$$

Igualamos la CDF a nuestro número aleatorio R:

$$R = 1 - e^{-\lambda X}$$

Despejamos X:

$$e^{-\lambda X} = 1 - R$$

$$-\lambda X = \ln(1 - R)$$

$$X = -\frac{1}{\lambda} \ln(1 - R)$$

Nota Técnica: Dado que si R es uniforme en $(0, 1)$, entonces $(1-R)$ también lo es, podemos simplificar la fórmula computacionalmente a:

$$X = -\frac{1}{\lambda} \ln(R) \quad \text{o} \quad X = -\text{media} \times \ln(R) \tag{4}$$

Paso 2: Ejecución Numérica Integrada. Utilicemos los números R_1, R_2, R_3 generados en el **Ejemplo 1** para simular tres tiempos de servicio consecutivos. Recordemos: media $= 0.5$.

- **Cliente 1 (usando $R_1 = 0.3750$):**

$$X_1 = -0.5 \times \ln(0.3750)$$

$$X_1 = -0.5 \times (-0.9808) = \mathbf{0.4904} \text{ minutos}$$

- **Cliente 2 (usando $R_2 = 0.0625$):**

$$X_2 = -0.5 \times \ln(0.0625)$$

$$X_2 = -0.5 \times (-2.7725) = \mathbf{1.3862} \text{ minutos}$$

  Interpretación: Un valor de R bajo genera un valor de X alto en la fórmula simplificada (o viceversa dependiendo de si usamos R o $1 - R$), capturando la "cola" de la distribución.

- **Cliente 3 (usando $R_3 = 0.5000$):**

$$X_3 = -0.5 \times \ln(0.5000)$$

$$X_3 = -0.5 \times (-0.6931) = \mathbf{0.3465} \text{ minutos}$$

Estos valores X_1, X_2, X_3 son ahora datos de entrada válidos para una simulación de eventos discretos.

4. Resumen: del LCG a la Variada

El recorrido de esta lectura puede sintetizarse en tres pasos sucesivos que forman la columna vertebral de cualquier generador estocástico:

Paso	Operación	Resultado
1	LCG: $Z_i = (aZ_{i-1} + c) \text{ mód } m$	Entero pseudoaleatorio $Z_i \in \{0, \ldots, m-1\}$
2	Normalización: $R_i = Z_i/m$	Número uniforme $R_i \in (0, 1)$
3	Transformada inversa: $X_i = F^{-1}(R_i)$	Variada con distribución F deseada

En la práctica, librerías como `numpy` implementan generadores mucho más sofisticados (Mersenne Twister, con período $2^{19937} - 1$) y transformadas inversas para decenas de distribuciones. Sin embargo, comprender el LCG y la Transformada Inversa permite entender *por qué* estas herramientas funcionan y cómo diagnosticar sus limitaciones. La implementación computacional completa de estos conceptos se trabaja en el notebook correspondiente del curso.

5. Conclusión

Hemos recorrido el camino completo desde la aritmética modular básica hasta la generación de variables estocásticas útiles para la toma de decisiones. Es crucial entender que la calidad de nuestra

simulación depende intrínsecamente de la calidad de nuestro generador $U(0,1)$ (el periodo del LCG) y de la precisión analítica de nuestra transformada inversa $F^{-1}(R)$.

Nota: cuando la CDF no tiene una inversa analítica simple (como en la distribución Normal) se aplican otros métodos como el de Aceptación-Rechazo y Box-Muller. Si este tema te interesa, te exhortamos a consultarlos.

Referencias

[1] Law, A. M. (2015). *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill. Cap. 8.

[2] Banks, J., Carson, J., Nelson, B., & Nicol, D. (2014). *Discrete-Event System Simulation*, 5th ed. Pearson. Cap. 8.

[3] Knuth, D. E. (1998). *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley. Cap. 3.

[4] Ross, S. M. (2006). *Simulation*, 4th ed. Academic Press. Cap. 5.

[5] L'Ecuyer, P. (1999). Good parameters and implementations for combined multiple recursive random number generators. *Operations Research*, 47(1), 159–164.